



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Estructuras de Selección
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	1 - B
Alumnos participantes:	Punina Moposita Kevin David
Asignatura:	Algoritmos y Lógica de Programación.
Docente:	Ing. Jose Caiza Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Utilizar estructuras de selección en el análisis y diseño de algoritmos.

Específicos:

- Implementar un programa en C++ y Java que permita registrar, mostrar y modificar una hora, aplicando una estructura repetitiva controlada por la respuesta del usuario para decidir cuándo finalizar el ciclo de cambios.
- Diseñar un sistema de cuentas bancarias orientado a objetos que relacione un cliente con hasta tres cuentas, permitiendo ejecutar operaciones de envío, recepción y transferencia de dinero con validación de fondos suficientes.
- Modelar mediante una clase Jean el estado físico de una prenda (tenidos, humedad y estado de la tela), implementando los métodos Lavar, Secar y MostrarDatos para reflejar el cambio de ese estado a través de las acciones del menú.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

El trabajo se desarrollará de manera individual.

- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y realice la codificación en C++.
- Subir en un documento en formato pdf que contenga capturas del código de cada ejercicio, adicionalmente, comprimir en una carpeta los archivos con el código generado, y subirlo al aula virtual.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- **Inteligencia artificial**
- **Laptop**
- **Papel**
- **Material de escritorio**

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales



- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas

2.7 Resultados obtenidos

Ejercicio 1: control de hora

Enunciado: Desarrollar un programa en C++ y java que permita ingresar una hora, minutos y segundos. Luego, el sistema debe mostrar la hora registrada y preguntar al usuario si desea cambiarla. Si responde que sí, se debe permitir modificar la hora y volver a mostrarla. Si responde que no, el programa finaliza. Este enunciado se basa en el flujo del programa en c++ y java donde se ingresan horas, minutos y segundos, se muestran y luego se consulta si se desea cambiar la hora.

Análisis.

El problema consiste en registrar una hora compuesta por hora, minutos y segundos, mostrar al usuario si desea modificarla o no.

Datos de entrada

Hora int
Minutos int
Segundos int
Respuesta si o no

Procesos

- Leer hora, minutos y segundos ingresados por el usuario
- Formatear y mostrar la hora registrada
- Preguntar si el usuario desea cambiar la hora
- Si la respuesta es afirmativa, leer nuevamente los tres valores y volver a mostrarlos
- Repetir la pregunta hasta que el usuario responda que no
- Finalizar el programa

Datos de salida

- Hora registrada en formato HH: MM: SS
- Finalizar programa

Algoritmo

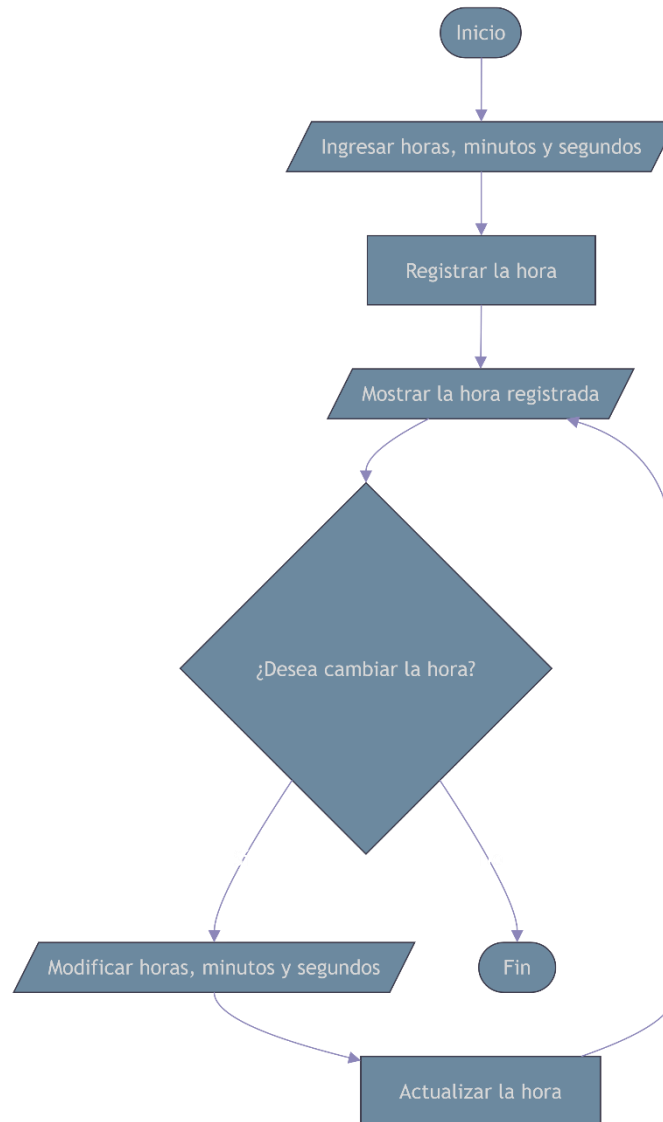
Primero se le pide al usuario los tres números que forman la hora. Apenas los tiene, el programa arma un texto con ceros a la izquierda cuando hace falta, por ejemplo 9 se muestra como 09, y lo despliega en pantalla como 'Hora registrada'.

Después entra en un ciclo que pregunta '¿Desea cambiar la hora?'.

Si el usuario responde que si, el programa vuelve a pedir hora, minutos y segundos. Muestra la hora actualizada.

Si responde que no, el ciclo se corta y el programa se despide.

Diagrama de flujo



Pseudocódigo

Proceso ControlDeHora

Definir hora, minutos, segundos Como Entero;

Definir respuesta Como Caracter;

Escribir "=== CONTROL DE HORA ===";

Escribir "Ingrese la hora (0-23):";

Leer hora;

Escribir "Ingrese los minutos (0-59):";

Leer minutos;

Escribir "Ingrese los segundos (0-59):";

Leer segundos;



Escribir "Hora registrada: ", hora, ":", minutos, ":", segundos;

Repetir

Escribir "Desea cambiar la hora? (s/n):";

Leer respuesta;

Si respuesta = "s" O respuesta = "S" Entonces

Escribir "Ingrese la nueva hora (0-23):";

Leer hora;

Escribir "Ingrese los nuevos minutos (0-59):";

Leer minutos;

Escribir "Ingrese los nuevos segundos (0-59):";

Leer segundos;

Escribir "Hora registrada: ", hora, ":", minutos, ":", segundos;

FinSi

Hasta Que respuesta = "n" O respuesta = "N"

Escribir "Programa finalizado.";

FinProceso

Prueba de escritorio

Paso	Instrucción	hora	minutos	segundos	respuesta	Salida en pantalla
1	Leer hora	8	-	-	-	-
2	Leer minutos	8	5	-	-	-
3	Leer segundos	8	5	0	-	-
4	Mostrar hora	8	5	0	-	Hora registrada: 08:05:00



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



Paso	Instrucción	hora	minutos	segundos	respuesta	Salida en pantalla
5	Leer respuesta	8	5	0	s	Desea cambiar la hora? (s/n):
6	Leer hora (nueva)	14	5	0	s	Ingrese la nueva hora (0-23):
7	Leer minutos (nuevo)	14	30	0	s	Ingrese los nuevos minutos (0-59):
8	Leer segundos (nuevo)	14	30	45	s	Ingrese los nuevos segundos (0-59):
9	Mostrar hora	14	30	45	s	Hora registrada: 14:30:45
10	Leer respuesta	14	30	45	n	Desea cambiar la hora? (s/n):
11	Evaluar condición del ciclo (respuesta = n)	14	30	45	n	Se sale del ciclo Repetir
12	Mostrar mensaje final	14	30	45	n	Programa finalizado.

Capturas códigos



```
1  import java.util.Scanner;
2
3  public class Ejercicio1 {
4
5      static void mostrarHora(int h, int m, int s) {
6          System.out.printf("Hora registrada: %02d:%02d:%02d\n", h, m, s);
7      }
8
9      public static void main(String[] args) {
10         Scanner sc = new Scanner(System.in);
11         int hora, minutos, segundos;
12         String respuesta;
13
14         System.out.println("=== CONTROL DE HORA ===");
15         System.out.print("Ingrese la hora (0-23): ");
16         hora = sc.nextInt();
17         System.out.print("Ingrese los minutos (0-59): ");
18         minutos = sc.nextInt();
19         System.out.print("Ingrese los segundos (0-59): ");
20         segundos = sc.nextInt();
21
22         mostrarHora(hora, minutos, segundos);
23
24         do {
25             System.out.print("Desea cambiar la hora? (s/n): ");
26             respuesta = sc.next();
27
28             if (respuesta.equalsIgnoreCase("s")) {
29                 System.out.print("Ingrese la nueva hora (0-23): ");
30                 hora = sc.nextInt();
31                 System.out.print("Ingrese los nuevos minutos (0-59): ");
32                 minutos = sc.nextInt();
33                 System.out.print("Ingrese los nuevos segundos (0-59): ");
34                 segundos = sc.nextInt();
35
36                 mostrarHora(hora, minutos, segundos);
37             }
38         } while (respuesta.equalsIgnoreCase("s"));
39
40         System.out.println("Programa finalizado.");
41         sc.close();
42     }
43 }
44 }
```

C++



```
1  #include <iostream>
2  using namespace std;
3
4  void mostrarHora(int h, int m, int s) {
5      cout << "Hora registrada: ";
6      if (h < 10) cout << "0";
7      cout << h << ":";
8      if (m < 10) cout << "0";
9      cout << m << ":";
10     if (s < 10) cout << "0";
11     cout << s << endl;
12 }
13
14 int main() {
15     int hora, minutos, segundos;
16     char respuesta;
17
18     cout << "=== CONTROL DE HORA ===" << endl;
19     cout << "Ingrese la hora (0-23): ";
20     cin >> hora;
21     cout << "Ingrese los minutos (0-59): ";
22     cin >> minutos;
23     cout << "Ingrese los segundos (0-59): ";
24     cin >> segundos;
25
26     mostrarHora(hora, minutos, segundos);
27
28     do {
29         cout << "Desea cambiar la hora? (s/n): ";
30         cin >> respuesta;
31
32         if (respuesta == 's' || respuesta == 'S') {
33             cout << "Ingrese la nueva hora (0-23): ";
34             cin >> hora;
35             cout << "Ingrese los nuevos minutos (0-59): ";
36             cin >> minutos;
37             cout << "Ingrese los nuevos segundos (0-59): ";
38             cin >> segundos;
39
40             mostrarHora(hora, minutos, segundos);
41         }
42     } while (respuesta == 's' || respuesta == 'S');
43
44     cout << "Programa finalizado." << endl;
45     return 0;
46 }
47
```



Ejercicio 2: Sistemas de cuentas bancarias

Enunciado: Desarrollar un programa en C++ y java que permita registrar un cliente mediante su DNI y crear hasta tres cuentas bancarias. El usuario podrá seleccionar una cuenta y realizar acciones desde un menú: ver atributos de la cuenta, enviar dinero, recibir dinero, salir del sistema o hacer una transferencia entre cuentas

Análisis.

El ejercicio pide manejar un cliente identificado por su DNI que puede tener hasta tres cuentas bancarias. Se necesitan dos entidades: el Cliente, que agrupa las cuentas, y la CuentaBancaria, que guarda su número y su saldo y sabe enviar y recibir dinero.

Datos de entrada

- DNI del cliente (texto)
- Cantidad de cuentas a registrar (entero, maximo 3)
- Número de cuenta y saldo inicial, por cada cuenta creada
- Cuenta seleccionada para operar (entero)
- Opción del menú (ver atributos, enviar, recibir, transferir, salir)

Procesos

- Registrar el cliente con su DNI
- Crear hasta tres cuentas bancarias asociadas al cliente
- Listar las cuentas disponibles y permitir seleccionar una
- Enviar dinero, descontando el saldo si hay fondos suficientes
- Recibir dinero, sumando el monto al saldo de la cuenta
- Transferir dinero entre dos cuentas del mismo cliente
- Repetir el menú hasta que el usuario elija salir

Datos de salida

- Listado de cuentas del cliente con número y saldo
- Confirmación de envío, recepción o transferencia de dinero
- Mensaje de fondos insuficientes cuando corresponda
- Mensaje de salida del sistema

Algoritmo

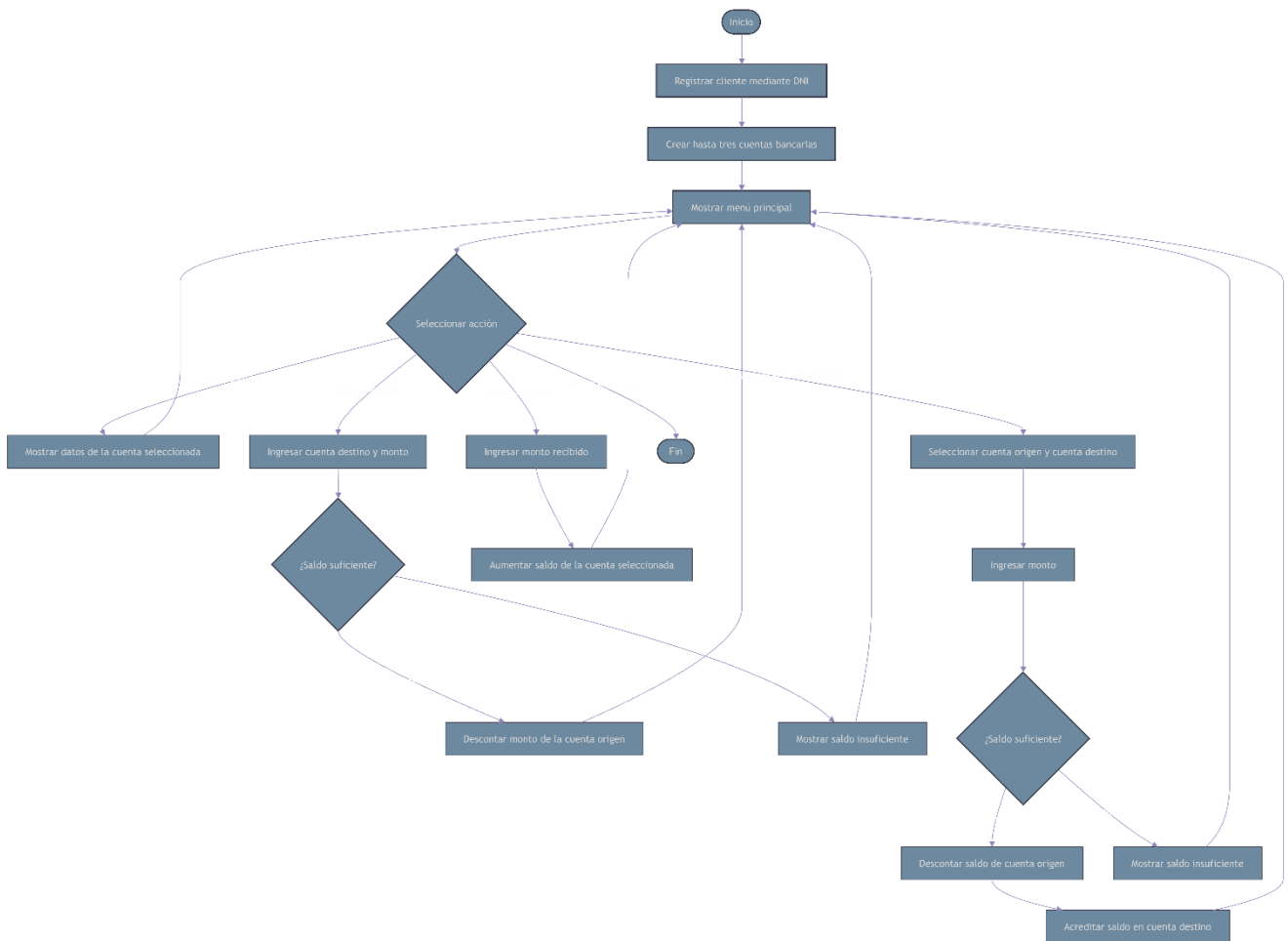
El programa arranca pidiendo el DNI del cliente y cuantas cuentas quiere abrir, sin pasarse de tres. Por cada cuenta pide un número y un saldo inicial, y las va guardando.

Después entra en un menú que se repite: primero pide elegir con que cuenta se va a trabajar, y luego muestra las cinco opciones disponibles. Si el usuario pide ver los atributos, simplemente se imprime el número y el saldo de esa cuenta. Si pide enviar dinero, se revisa que el saldo alcance antes de restar el monto; si no alcanza, se avisa que los fondos son insuficientes y no se descuenta nada. Si pide recibir dinero, el monto se suma sin condiciones.

La transferencia es la operación que junta las dos cuentas: se pide una cuenta destino distinta de la actual, se intenta enviar el dinero desde la cuenta de origen y, solo si ese envío tuvo éxito, se le suma el mismo monto a la cuenta destino. Esto evita que el dinero desaparezca o se duplique si no hay fondos suficientes. El programa sigue mostrando el menú hasta que el usuario elige la opción de salir.



Diagrama de flujo



Pseudocódigo

Proceso SistemaCuentasBancarias

Definir dni Como Caracter;

Definir cantidadCuentas, i, seleccion, destino, opcion Como Entero;

Definir numero Como Caracter;

Definir numeroCuentas[3] Como Caracter;

Definir saldoCuentas[3] Como Real;

Definir monto Como Real;

Escribir "=== SISTEMA DE CUENTAS BANCARIAS ===";

Escribir "Ingrese el DNI del cliente:";

Leer dni;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



```
Escribir "Cuántas cuentas desea registrar (máximo 3)?";

Leer cantidadCuentas;

Si cantidadCuentas > 3 Entonces
    cantidadCuentas <- 3;
FinSi

Para i <- 1 Hasta cantidadCuentas Hacer
    Escribir "Número de la cuenta ", i;
    Leer numeroCuentas[i];
    Escribir "Saldo inicial:";
    Leer saldoCuentas[i];
FinPara

opcion <- 0;

Mientras opcion <> 5 Hacer
    Escribir "Seleccione una cuenta (1-", cantidadCuentas, "):";
    Leer seleccion;

    Escribir "1. Ver atributos de la cuenta";
    Escribir "2. Enviar dinero";
    Escribir "3. Recibir dinero";
    Escribir "4. Transferencia entre cuentas";
    Escribir "5. Salir";
    Leer opcion;

Segun opcion Hacer
    1:
        Escribir "Cuenta: ", numeroCuentas[seleccion], " Saldo: ",
saldoCuentas[seleccion];
    2:
        Escribir "Monto a enviar:";
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



Leer monto;

Si monto <= saldoCuentas[seleccion] Entonces

saldoCuentas[seleccion] <- saldoCuentas[seleccion] - monto;

Sino

Escribir "Fondos insuficientes";

FinSi

3:

Escribir "Monto a recibir:";

Leer monto;

saldoCuentas[seleccion] <- saldoCuentas[seleccion] + monto;

4:

Escribir "Cuenta destino (1-", cantidadCuentas, "):";

Leer destino;

Escribir "Monto a transferir:";

Leer monto;

Si monto <= saldoCuentas[seleccion] Entonces

saldoCuentas[seleccion] <- saldoCuentas[seleccion] - monto;

saldoCuentas[destino] <- saldoCuentas[destino] + monto;

Sino

Escribir "Fondos insuficientes";

FinSi

5:

Escribir "Saliendo del sistema...";

FinSegun

FinMientras

FinProceso

Prueba de Escritorio.

Paso	Acción	Cuenta sel.	Opción	Monto	Saldo cta. 001	Saldo cta. 002	Salida en pantalla
1	Leer DNI del cliente	-	-	-	-	-	-



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



Paso	Acción	Cuenta sel.	Opción	Monto	Saldo cta. 001	Saldo cta. 002	Salida en pantalla
2	Leer cantidad de cuentas	-	-	-	-	-	cantidadCuentas = 2
3	Crear cuenta 1 (numero=001, saldo=500)	-	-	-	500	-	-
4	Crear cuenta 2 (numero=002, saldo=200)	-	-	-	500	200	-
5	Seleccionar cuenta	001	-	-	500	200	Seleccione una cuenta (1-2):
6	Elegir opción 2 (enviar dinero)	001	2	100	500	200	Monto a enviar:
7	Restar monto de la cuenta 001	001	2	100	400	200	Se enviaron \$100 desde la cuenta 001
8	Seleccionar cuenta	001	-	-	400	200	Seleccione una cuenta (1-2):
9	Elegir opción 4 (transferencia), destino=002	001	4	150	400	200	Monto a transferir:
10	Restar de 001 y sumar a 002	001	4	150	250	350	Se enviaron \$150 desde la cuenta 001 / Se recibieron \$150 en la cuenta 002
11	Seleccionar cuenta	002	-	-	250	350	Seleccione una cuenta (1-2):
12	Elegir opción 1 (ver atributos)	002	1	-	250	350	Cuenta N: 002 Saldo: \$350
13	Elegir opción 5 (salir)	002	5	-	250	350	Saliendo del sistema...

**Códigos
JAVA**



```
1  import java.util.Scanner;
2
3  class CuentaBancaria {
4      private String numero;
5      private double saldo;
6
7      public CuentaBancaria() {
8          this.numero = "";
9          this.saldo = 0.0;
10     }
11
12     public CuentaBancaria(String numero, double saldoInicial) {
13         this.numero = numero;
14         this.saldo = saldoInicial;
15     }
16
17     public String getNumero() { return numero; }
18     public double getSaldo() { return saldo; }
19
20     public void mostrarDatos() {
21         System.out.println("Cuenta N: " + numero + " | Saldo: $" + saldo);
22     }
23
24     public void recibirDinero(double monto) {
25         saldo += monto;
26         System.out.println("Se recibieron $" + monto + " en la cuenta " + numero);
27     }
28
29     public boolean enviarDinero(double monto) {
30         if (monto > saldo) {
31             System.out.println("Fondos insuficientes en la cuenta " + numero);
32             return false;
33         }
34         saldo -= monto;
35         System.out.println("Se enviaron $" + monto + " desde la cuenta " + numero);
36         return true;
37     }
38 }
39
40 class Cliente {
41     private String dni;
42     private CuentaBancaria[] cuentas;
43     private int cantidadCuentas;
44
45     public Cliente(String dni) {
46         this.dni = dni;
47         this.cuentas = new CuentaBancaria[3];
48         this.cantidadCuentas = 0;
49     }
50
51     public String getDni() { return dni; }
52     public int getCantidadCuentas() { return cantidadCuentas; }
53
54     public void crearCuenta(String numero, double saldoInicial) {
55         if (cantidadCuentas < 3) {
56             cuentas[cantidadCuentas] = new CuentaBancaria(numero, saldoInicial);
57             cantidadCuentas++;
58         } else {
59             System.out.println("El cliente ya tiene el maximo de 3 cuentas.");
60         }
61     }
62
63     public CuentaBancaria getCuenta(int indice) {
64         if (indice >= 0 && indice < cantidadCuentas) {
65             return cuentas[indice];
66         }
67         return null;
68     }
69 }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



```
70     public void listarCuentas() {
71         for (int i = 0; i < cantidadCuentas; i++) {
72             System.out.print((i + 1) + ". ");
73             cuentas[i].mostrarDatos();
74         }
75     }
76 }
77
78 public class Ejercicio2 {
79     public static void main(String[] args) {
80         Scanner sc = new Scanner(System.in);
81
82         System.out.println("=== SISTEMA DE CUENTAS BANCARIAS ===");
83         System.out.print("Ingrese el DNI del cliente: ");
84         String dni = sc.next();
85
86         Cliente cliente = new Cliente(dni);
87
88         System.out.print("Cuantas cuentas desea registrar (maximo 3)? ");
89         int cantidad = sc.nextInt();
90         if (cantidad > 3) cantidad = 3;
91
92         for (int i = 0; i < cantidad; i++) {
93             System.out.print("Numero de la cuenta " + (i + 1) + ": ");
94             String numero = sc.next();
95             System.out.print("Saldo inicial: ");
96             double saldoInicial = sc.nextDouble();
97             cliente.crearCuenta(numero, saldoInicial);
98         }
99
100        int opcion = -1;
101        while (opcion != 5) {
102            System.out.println("\nCliente DNI: " + cliente.getDni());
103            cliente.listarCuentas();
104
105            System.out.print("\nSeleccione una cuenta (1-" + cliente.getCantidadCuentas() + "): ");
106            int seleccion = sc.nextInt() - 1;
107
108            CuentaBancaria cuenta = cliente.getCuenta(seleccion);
109            if (cuenta == null) {
110                System.out.println("Cuenta invalida.");
111                continue;
112            }
113
114            System.out.println("\n--- MENU ---");
115            System.out.println("1. Ver atributos de la cuenta");
116            System.out.println("2. Enviar dinero");
117            System.out.println("3. Recibir dinero");
118            System.out.println("4. Transferencia entre cuentas");
119            System.out.println("5. Salir");
120            System.out.print("Opcion: ");
121            opcion = sc.nextInt();
122
123            switch (opcion) {
124                case 1:
125                    cuenta.mostrarDatos();
126                    break;
127                case 2: {
128                    System.out.print("Monto a enviar: ");
129                    double monto = sc.nextDouble();
130                    cuenta.enviarDinero(monto);
131                    break;
132                }
133                case 3: {
134                    System.out.print("Monto a recibir: ");
135                    double monto = sc.nextDouble();
136                    cuenta.recibirDinero(monto);
137                    break;
138                }
139                case 4: {
```



```
139         case 4: {
140             System.out.print("Seleccione cuenta destino (1-" + cliente.getCantidadCuentas() + "):");
141             int destino = sc.nextInt() - 1;
142             CuentaBancaria cuentaDestino = cliente.getCuenta(destino);
143             if (cuentaDestino == null || cuentaDestino == cuenta) {
144                 System.out.println("Cuenta destino invalida.");
145                 break;
146             }
147             System.out.print("Monto a transferir: ");
148             double monto = sc.nextDouble();
149             if (cuenta.enviarDinero(monto)) {
150                 cuentaDestino.recibirDinero(monto);
151             }
152             break;
153         }
154         case 5:
155             System.out.println("Saliendo del sistema...");
156             break;
157         default:
158             System.out.println("Opcion invalida.");
159     }
160 }
161
162 sc.close();
163 }
164 }
```

C++

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class CuentaBancaria {
6  private:
7      string numero;
8      double saldo;
9
10 public:
11     CuentaBancaria() {
12         numero = "";
13         saldo = 0.0;
14     }
15
16     CuentaBancaria(string num, double saldoInicial) {
17         numero = num;
18         saldo = saldoInicial;
19     }
20
21     string getNumero() { return numero; }
22     double getSaldo() { return saldo; }
23
24     void mostrarDatos() {
25         cout << "Cuenta N: " << numero << " | Saldo: $" << saldo << endl;
26     }
27
28     void recibirDinero(double monto) {
29         saldo += monto;
30         cout << "Se recibieron $" << monto << " en la cuenta " << numero << endl;
31     }
32
33     bool enviarDinero(double monto) {
34         if (monto > saldo) {
35             cout << "Fondos insuficientes en la cuenta " << numero << endl;
36             return false;
37         }
38         saldo -= monto;
39         cout << "Se enviaron $" << monto << " desde la cuenta " << numero << endl;
40         return true;
41     }
42 };
43
44 class Cliente {
45 private:
46     string dni;
47     CuentaBancaria cuentas[3];
48     int cantidadCuentas;
49
50 public:
51     Cliente(string documento) {
52         dni = documento;
```



```
52     dni = documento;
53     cantidadCuentas = 0;
54 }
55
56 string getDni() { return dni; }
57 int getCantidadCuentas() { return cantidadCuentas; }
58
59 void crearCuenta(string numero, double saldoInicial) {
60     if (cantidadCuentas < 3) {
61         cuentas[cantidadCuentas] = CuentaBancaria(numero, saldoInicial);
62         cantidadCuentas++;
63     } else {
64         cout << "El cliente ya tiene el maximo de 3 cuentas." << endl;
65     }
66 }
67
68 CuentaBancaria* getCuenta(int indice) {
69     if (indice >= 0 && indice < cantidadCuentas) {
70         return &cuentas[indice];
71     }
72     return nullptr;
73 }
74
75 void listarCuentas() {
76     for (int i = 0; i < cantidadCuentas; i++) {
77         cout << i + 1 << ". ";
78         cuentas[i].mostrarDatos();
79     }
80 }
81 };
82
83 int main() {
84     string dni;
85     cout << "=== SISTEMA DE CUENTAS BANCARIAS ===" << endl;
86     cout << "Ingrese el DNI del cliente: ";
87     cin >> dni;
88
89     Cliente cliente(dni);
90
91     int cantidad;
92     cout << "Cuantas cuentas desea registrar (maximo 3)? ";
93     cin >> cantidad;
94     if (cantidad > 3) cantidad = 3;
95
96     for (int i = 0; i < cantidad; i++) {
97         string numero;
98         double saldoInicial;
99         cout << "Numero de la cuenta " << i + 1 << ": ";
100        cin >> numero;
101        cout << "Saldo inicial: ";
102        cin >> saldoInicial;
103        cliente.crearCuenta(numero, saldoInicial);
104    }
```



```
118         cout << "Cuenta invalida." << endl;
119         continue;
120     }
121
122     cout << "\n--- MENU ---" << endl;
123     cout << "1. Ver atributos de la cuenta" << endl;
124     cout << "2. Enviar dinero" << endl;
125     cout << "3. Recibir dinero" << endl;
126     cout << "4. Transferencia entre cuentas" << endl;
127     cout << "5. Salir" << endl;
128     cout << "Opcion: ";
129     cin >> opcion;
130
131     double monto;
132     switch (opcion) {
133     case 1:
134         cuenta->mostrarDatos();
135         break;
136     case 2:
137         cout << "Monto a enviar: ";
138         cin >> monto;
139         cuenta->enviarDinero(monto);
140         break;
141     case 3:
142         cout << "Monto a recibir: ";
143         cin >> monto;
144         cuenta->recibirDinero(monto);
145         break;
146     case 4: {
147         cout << "Seleccione cuenta destino (1-" << cliente.getCantidadCuentas() << "): ";
148         int destino;
149         cin >> destino;
150         destino--;
151         CuentaBancaria* cuentaDestino = cliente.getCuenta(destino);
152         if (cuentaDestino == nullptr || cuentaDestino == cuenta) {
153             cout << "Cuenta destino invalida." << endl;
154             break;
155         }
156         cout << "Monto a transferir: ";
157         cin >> monto;
158         if (cuenta->enviarDinero(monto)) {
159             cuentaDestino->recibirDinero(monto);
160         }
161         break;
162     }
163     case 5:
164         cout << "Saliendo del sistema..." << endl;
165         break;
166     default:
167         cout << "Opcion invalida." << endl;
168     }
169 }
```

Ejercicio 3: GESTIÓN DE UN JEAN EN FÁBRICA O TIENDA

Enunciado: Desarrollar un programa en C++ y java que permita registrar los datos de un jean: código, color, talla, si fue teñido, cantidad de teñidos, precio, cantidad de botones, humedad y estado de la tela. El sistema debe tener opciones para mostrar los datos del jean, lavarlo y secarlo. Al lavar, disminuye la cantidad de teñidos; al secar, disminuye la humedad. Este enunciado se basa en la clase Jean.java, que contiene esos atributos y los métodos Lavar, Secar y MostrarDatos.

Análisis.

El objeto central es el jean, con un conjunto amplio de atributos que describen tanto su identidad (código, color, talla, precio, botones) como su estado actual en el proceso de fabricacion o cuidado (si fue teñido, cuantos teñidos lleva, su humedad y el estado de la tela). El programa modela dos acciones que cambian ese estado con el tiempo: lavar y secar.

Datos de entrada

- Código del jean (texto)



- Color (texto)
- Talla (texto)
- Si fue tenido (s/n)
- Cantidad de tenidos (entero)
- Precio (real)
- Cantidad de botones (entero)
- Humedad actual, en porcentaje (real)
- Estado de la tela (texto)
- Opción del menú (mostrar datos, lavar, secar, salir)

Procesos

- Registrar todos los atributos del jean al iniciar el programa
- Mostrar los datos del jean cuando se solicite
- Lavar el jean: disminuir la cantidad de tenidos y aumentar la humedad
- Secar el jean: disminuir la humedad
- Evitar que la humedad baje de 0 o suba de 100
- Repetir el menú hasta que el usuario elija salir

Datos de salida

- Datos completos del jean (codigo, color, talla, tenido, tenidos, precio, botones, humedad, estado de la tela)
- Mensaje de confirmación al lavar, con los tenidos restantes
- Mensaje de confirmación al secar, con la humedad actual
- Mensaje de salida del programa

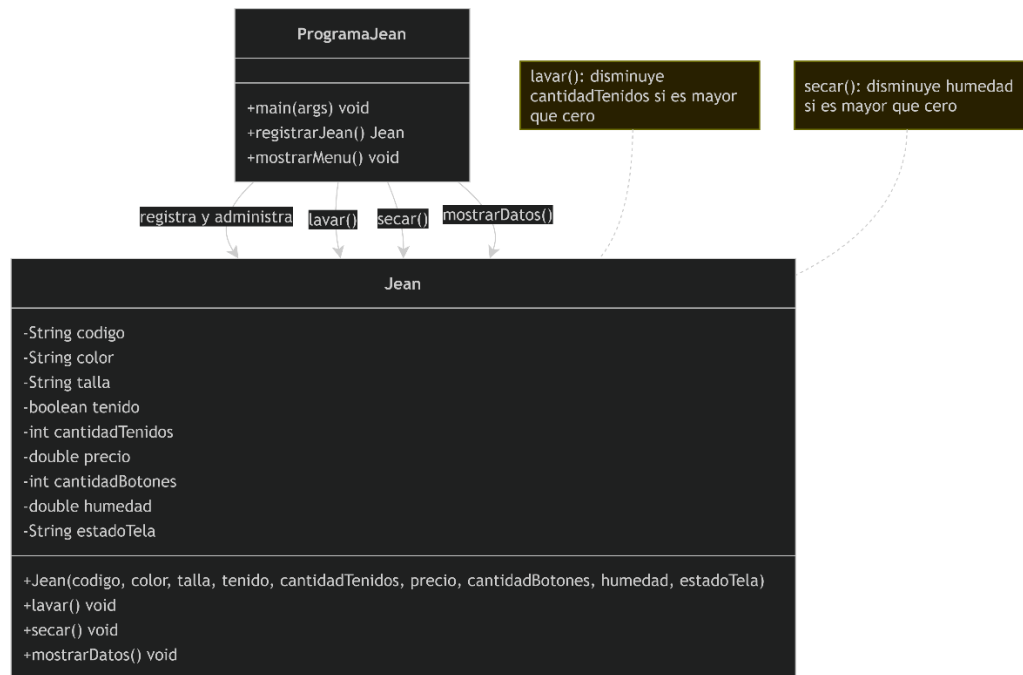
Algoritmo

El programa empieza pidiendo todos los datos del jean, uno por uno, y los guarda como el estado inicial de la prenda. A partir de ahí se despliega un menú con cuatro opciones que se repite hasta que el usuario decide salir.

Si el usuario elige mostrar datos, simplemente se imprimen todos los atributos tal como están en ese momento. Si elige lavar, el programa revisa si todavía le quedan tenidos por gastar: si es así, resta uno y sube la humedad, cuidando que nunca pase de 100. Si ya no hay tenidos, avisa que no hay más que perder.

Si el usuario elige secar, el programa revisa si hay humedad; si la hay, la reduce, cuidando que no baje de 0, y si ya está en 0 avisa que el jean ya está seco. De esta forma, lavar y secar son acciones opuestas sobre la misma prenda que van modificando su estado con cada uso, y el programa siempre refleja el estado más reciente cuando se piden los datos.

Diagrama de flujo



Pseudocódigo

Proceso GestionJean

Definir codigo, color, talla, estadoTela, respuestaTenido Como Caracter;

Definir tenido Como Logico;

Definir cantidadTenidos, cantidadBotones, opcion Como Entero;

Definir precio, humedad Como Real;

Escribir "=== REGISTRO DE JEAN ===";

Escribir "Codigo:";

Leer codigo;

Escribir "Color:";

Leer color;

Escribir "Talla:";

Leer talla;

Escribir "Fue tenido? (s/n):";

Leer respuestaTenido;

tenido <- (respuestaTenido = "s" O respuestaTenido = "S");

Escribir "Cantidad de tenidos:";

Leer cantidadTenidos;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



Escribir "Precio:";

Leer precio;

Escribir "Cantidad de botones:";

Leer cantidadBotones;

Escribir "Humedad actual (%):";

Leer humedad;

Escribir "Estado de la tela:";

Leer estadoTela;

Repetir

Escribir "1. Mostrar datos";

Escribir "2. Lavar";

Escribir "3. Secar";

Escribir "4. Salir";

Leer opcion;

Segun opcion Hacer

1:

Escribir "Codigo: ", codigo;

Escribir "Color: ", color;

Escribir "Talla: ", talla;

Escribir "Tenido: ", tenido;

Escribir "Cantidad de tenidos: ", cantidadTenidos;

Escribir "Precio: ", precio;

Escribir "Cantidad de botones: ", cantidadBotones;

Escribir "Humedad: ", humedad;

Escribir "Estado de la tela: ", estadoTela;

2:

Si cantidadTenidos > 0 Entonces

cantidadTenidos <- cantidadTenidos - 1;

humedad <- humedad + 20;

Si humedad > 100 Entonces



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



humedad <- 100;

FinSi

Sino

Escribir "El jean ya no tiene tenidos que perder";

FinSi

3:

Si humedad > 0 Entonces

humedad <- humedad - 25;

Si humedad < 0 Entonces

humedad <- 0;

FinSi

Sino

Escribir "El jean ya esta seco";

FinSi

4:

Escribir "Programa finalizado.";

FinSegun

Hasta Que opcion = 4

FinProceso

Prueba de escritorio

Paso	Opción elegida	cantidadTenidos	humedad	Salida en pantalla
1	Registro inicial del jean	3	40	-
2	1. Mostrar datos	3	40	Codigo: J001 ... Cantidad de tenidos: 3 ... Humedad: 40%
3	2. Lavar	2	60	El jean fue lavado. Tenidos restantes: 2
4	2. Lavar	1	80	El jean fue lavado. Tenidos restantes: 1
5	3. Secar	1	55	El jean fue secado. Humedad actual: 55%
6	1. Mostrar datos	1	55	Cantidad de tenidos: 1 ... Humedad: 55%



Paso	Opción elegida	cantidadTenidos	humedad	Salida en pantalla
7	4. Salir	1	55	Programa finalizado.

Códigos

JAVA

```
1  import java.util.Scanner;
2
3  class Jean {
4      private String codigo;
5      private String color;
6      private String talla;
7      private boolean tenido;
8      private int cantidadTenidos;
9      private double precio;
10     private int cantidadBotones;
11     private double humedad;
12     private String estadoTela;
13
14     public Jean(String codigo, String color, String talla, boolean tenido, int cantidadTenidos,
15                 double precio, int cantidadBotones, double humedad, String estadoTela) {
16         this.codigo = codigo;
17         this.color = color;
18         this.talla = talla;
19         this.tenido = tenido;
20         this.cantidadTenidos = cantidadTenidos;
21         this.precio = precio;
22         this.cantidadBotones = cantidadBotones;
23         this.humedad = humedad;
24         this.estadoTela = estadoTela;
25     }
26
27     public void mostrarDatos() {
28         System.out.println("\n--- DATOS DEL JEAN ---");
29         System.out.println("Codigo: " + codigo);
30         System.out.println("Color: " + color);
31         System.out.println("Talla: " + talla);
32         System.out.println("Tenido: " + (tenido ? "Si" : "No"));
33         System.out.println("Cantidad de tenidos: " + cantidadTenidos);
34         System.out.println("Precio: $" + precio);
35         System.out.println("Cantidad de botones: " + cantidadBotones);
36         System.out.println("Humedad: " + humedad + "%");
37         System.out.println("Estado de la tela: " + estadoTela);
38     }
39 }
```



```
38     }
39
40     public void lavar() {
41         if (cantidadTenidos > 0) {
42             cantidadTenidos--;
43             humedad += 20;
44             if (humedad > 100) humedad = 100;
45             System.out.println("El jean fue lavado. Tenidos restantes: " + cantidadTenidos);
46         } else {
47             System.out.println("El jean ya no tiene tenidos que perder.");
48         }
49     }
50
51     public void secar() {
52         if (humedad > 0) {
53             humedad -= 25;
54             if (humedad < 0) humedad = 0;
55             System.out.println("El jean fue secado. Humedad actual: " + humedad + "%");
56         } else {
57             System.out.println("El jean ya esta seco.");
58         }
59     }
60 }
61
62 public class Ejercicio3 {
63     public static void main(String[] args) {
64         Scanner sc = new Scanner(System.in);
65
66         System.out.println("=== REGISTRO DE JEAN ===");
67         System.out.print("Codigo: ");
68         String codigo = sc.next();
69         System.out.print("Color: ");
70         String color = sc.next();
71         System.out.print("Talla: ");
72         String talla = sc.next();
```



```
73      System.out.print("Fue tenido? (s/n): ");
74      String respuestaTenido = sc.next();
75      boolean tenido = respuestaTenido.equalsIgnoreCase("s");
76      System.out.print("Cantidad de tenidos: ");
77      int cantidadTenidos = sc.nextInt();
78      System.out.print("Precio: ");
79      double precio = sc.nextDouble();
80      System.out.print("Cantidad de botones: ");
81      int cantidadBotones = sc.nextInt();
82      System.out.print("Humedad actual (%): ");
83      double humedad = sc.nextDouble();
84      System.out.print("Estado de la tela: ");
85      String estadoTela = sc.next();
86
87      Jean jean = new Jean(codigo, color, talla, tenido, cantidadTenidos, precio,
88      |      cantidadBotones, humedad, estadoTela);
89
90      int opcion;
91      do {
92          System.out.println("\n--- MENU JEAN ---");
93          System.out.println("1. Mostrar datos");
94          System.out.println("2. Lavar");
95          System.out.println("3. Secar");
96          System.out.println("4. Salir");
97          System.out.print("Opcion: ");
98          opcion = sc.nextInt();
99
100         switch (opcion) {
101             case 1:
102                 jean.mostrarDatos();
103                 break;
104             case 2:
105                 jean.lavar();
106                 break;
107
108                 break;
109             case 3:
110                 jean.secar();
111                 break;
112             case 4:
113                 System.out.println("Programa finalizado.");
114                 break;
115             default:
116                 System.out.println("Opcion invalida.");
117         }
118     } while (opcion != 4);
119
120     sc.close();
121 }
```

C++



```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Jean {
6  private:
7      string codigo;
8      string color;
9      string talla;
10     bool tenido;
11     int cantidadTenidos;
12     double precio;
13     int cantidadBotones;
14     double humedad;
15     string estadoTela;
16
17 public:
18     Jean(string cod, string col, string tal, bool ten, int cantTen, double prec,
19         int botones, double hum, string estado) {
20         codigo = cod;
21         color = col;
22         talla = tal;
23         tenido = ten;
24         cantidadTenidos = cantTen;
25         precio = prec;
26         cantidadBotones = botones;
27         humedad = hum;
28         estadoTela = estado;
29     }
30
31     void mostrarDatos() {
32         cout << "\n--- DATOS DEL JEAN ---" << endl;
33         cout << "Codigo: " << codigo << endl;
34         cout << "Color: " << color << endl;
35         cout << "Talla: " << talla << endl;
36         cout << "Tenido: " << (tenido ? "Si" : "No") << endl;
37         cout << "Cantidad de tenidos: " << cantidadTenidos << endl;
38         cout << "Precio: $" << precio << endl;
39         cout << "Cantidad de botones: " << cantidadBotones << endl;
40         cout << "Humedad: " << humedad << "%" << endl;
41         cout << "Estado de la tela: " << estadoTela << endl;
42     }
43
44     void lavar() {
45         if (cantidadTenidos > 0) {
46             cantidadTenidos--;
47             humedad += 20;
48             if (humedad > 100) humedad = 100;
49             cout << "El jean fue lavado. Tenidos restantes: " << cantidadTenidos << endl;
50         } else {
51             cout << "El jean ya no tiene tenidos que perder." << endl;
52         }
53     }
54 }
```



```
53 }
54
55 void secar() {
56     if (humedad > 0) {
57         humedad -= 25;
58         if (humedad < 0) humedad = 0;
59         cout << "El jean fue secado. Humedad actual: " << humedad << "%" << endl;
60     } else {
61         cout << "El jean ya esta seco." << endl;
62     }
63 }
64 };
65
66 int main() {
67     string codigo, color, talla, estadoTela, respuestaTenido;
68     int cantidadTenidos, cantidadBotones;
69     double precio, humedad;
70     bool tenido;
71
72     cout << "=== REGISTRO DE JEAN ===" << endl;
73     cout << "Codigo: ";
74     cin >> codigo;
75     cout << "Color: ";
76     cin >> color;
77     cout << "Talla: ";
78     cin >> talla;
79     cout << "Fue tenido? (s/n): ";
80     cin >> respuestaTenido;
81     tenido = (respuestaTenido == "s" || respuestaTenido == "S");
82     cout << "Cantidad de tenidos: ";
83     cin >> cantidadTenidos;
84     cout << "Precio: ";
85     cin >> precio;
86     cout << "Cantidad de botones: ";
87     cin >> cantidadBotones;
88     cout << "Humedad actual (%): ";
89     cin >> humedad;
90     cout << "Estado de la tela: ";
91     cin >> estadoTela;
92
93     Jean jean(codigo, color, talla, tenido, cantidadTenidos, precio, cantidadBotones, humedad, estadoTela);
94
95     int opcion;
96     do {
97         cout << "\n--- MENU JEAN ---" << endl;
98         cout << "1. Mostrar datos" << endl;
99         cout << "2. Lavar" << endl;
100        cout << "3. Secar" << endl;
101        cout << "4. Salir" << endl;
102        cout << "Opcion: ";
103        cin >> opcion;
104
105        switch (opcion) {
106            case 1:
107                jean.mostrarDatos();
108                break;
109            case 2:
110                jean.lavar();
111                break;
112            case 3:
113                jean.secar();
114                break;
115            case 4:
116                cout << "Programa finalizado." << endl;
117                break;
118            default:
119                cout << "Opcion invalida." << endl;
120        }
121
122    } while (opcion != 4);
123
124    return 0;
125 }
```



2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

Los tres ejercicios muestran que un mismo problema se resuelve con la misma lógica en C++ y en Java; lo que cambia es la sintaxis de entrada y salida (cin/cout frente a Scanner/System.out) y el manejo de memoria, mientras la estructura de control (ciclos, condicionales y menús) se mantiene igual en ambos lenguajes.

El ejercicio del control de hora confirma que separar la acción de mostrar datos en una función propia evita duplicar código y facilita que el programa se mantenga consistente cada vez que el estado cambia.

2.10 Recomendaciones

Agregar validación de rangos en la entrada de datos (por ejemplo, que la hora esté entre 0 y 23, o que la humedad no se ingrese fuera de 0 a 100) para evitar estados inválidos desde el inicio del programa.

Ampliar las pruebas más allá del caso normal, incluyendo casos límite como enviar dinero sin fondos suficientes, lavar un jean sin tenidos restantes o secar un jean con humedad en cero, para verificar que los mensajes de control funcionan correctamente.

Separar la lógica de negocio de la entrada y salida por consola (por ejemplo, usando una capa de menú independiente de las clases Jean, Cliente y CuentaBancaria).

2.11 Referencias bibliográficas

- [1] Coddy Tech, «Aprende C++ - Gratis Interactivo Curso», *Coddy*. <https://coddy.tech/landing/es/cpp>
- [2] Codecademy, «Learn C++», *Codecademy*. <https://www.codecademy.com/learn/learn-c-plus-plus>
- [3] CodeGym, «Aprende Java en línea de manera divertida», *CodeGym*. <https://codegym.cc/es/>
- [4] LearnJavaOnline.org, «Learn Java - Free Interactive Java Tutorial», *LearnJavaOnline.org*. <https://www.learnjavaonline.org>

2.12 Anexos